

Тепловой баланс котла

Подробная техническая документация

Десктопное приложение для расчёта теплового баланса
парового котла по формулам главы 5 (5-01 — 5-24)

Версия приложения: 1.0.0

Платформа: Windows 10 / 11 · Python 3

Стек: CustomTkinter · IAPWS-IF97 · matplotlib

Документ объединяет руководство пользователя,
описание архитектуры, формул, API, плагинов и справочных данных.

Содержание

1. Назначение и возможности
2. Установка и запуск
3. Руководство пользователя
4. Архитектура системы
5. Формулы теплового баланса
6. Программный API ядра
7. Система плагинов
8. Справочные данные и форматы файлов
9. Зависимости и структура проекта
10. Развитие и ограничения

1. Назначение и возможности

Приложение «Тепловой баланс котла» предназначено для инженерного расчёта теплового баланса парового котла. Расчёты выполняются по формулам главы 5 учебника (номера 5-01 — 5-24): располагаемое тепло топлива, статьи потерь, КПД котла, полезная тепловая мощность и расход топлива.

Основные возможности

- Расчёт располагаемого тепла $Q_{\text{дп}}$, потерь $q_{\text{пл}} \dots q_{\text{охл}}$ и $q_{\text{охл}}$, суммы потерь Σq , КПД η
- Расчёт полезного тепла $Q_{\text{полн}}$ и расходов топлива B , $B_{\text{н}}$
- Интерполяция номинальных потерь в окружающую среду $q_{\text{пл}}$ по рис. 5.1
- Справочник топлив на JSON (15 пресетов, расширяемый)
- Система плагинов — сторонние утилиты из каталога `plugins/`
- Калькулятор свойств воды и пара по стандарту IAPWS-IF97 с подстановкой энтальпий в баланс
- Сохранение и загрузка сессий расчёта в JSON
- Практические рекомендации по результатам (анализ режима)
- Модульное ядро `src/core` — использование без графического интерфейса

Для кого предназначено

Для студентов теплоэнергетических специальностей, инженеров-энергетиков и расчётчиков котельных установок, которым нужен прозрачный расчёт по учебным формулам с сохранением исходных данных и возможностью встраивания ядра в другие программные системы.

2. Установка и запуск

Требования

- Python 3 (с поддержкой tkinter — обычно есть в установке на Windows)
- Windows 10/11 рекомендуется (есть оптимизации DPI и перетаскивания окна)
- `pip` для установки зависимостей

Установка зависимостей

```
pip install -r requirements.txt
```

Пакет	Версия	Назначение
customtkinter	≥ 5.2.0	Современный GUI на базе tkinter
Pillow	≥ 10.0.0	Отрисовка логотипа
iapws	≥ 1.5.0	Свойства воды/пара IAPWS-IF97
matplotlib	≥ 3.8.0	Диаграмма потерь в результатах

Запуск

```
python main.py
```

Альтернатива на Windows: двойной клик по файлу `run.bat` (при ошибке окно консоли не закроется сразу).

Последовательность при старте

- Корень проекта добавляется в `sys.path`
- Применяются Windows-фиксы (`src/ui/win32_fix.py`)
- Создаётся `JsonFileDataProvider` для `data/reference/`
- Загружаются плагины из `plugins/` через `PluginRegistry`
- Запускается главное окно `HeatBalanceApp`

3. Руководство пользователя

Интерфейс построен на CustomTkinter в тёмной промышленной палитре. Слева — навигация по шести разделам, справа — поля ввода или результаты.

3.1. Топливо

- Выбор пресета из справочника (газ, мазут, угли и др.) либо ручной ввод
- Тип топлива: твёрдое/жидкое или газообразное — влияет на формулу $Q_{\text{п}}$ (5-02 / 5-02a)
- Параметры: $Q_{\text{р}}$, $A_{\text{р}}$, влажность, теплоёмкость $c_{\text{пл}}$, $(\text{CO}_{\text{пл}})_{\text{карб}}$
- Опция учёта физического тепла топлива $i_{\text{пл}} = c_{\text{пл}} \cdot t_{\text{пл}}$
- Параметры шлака и уноса для $q_{\text{пл}}$ и $q_{\text{пл}}'$: доли золы, содержание горючего, температура шлака

3.2. Уходящие газы

- Энтальпии $I_{\text{ух}}$, $I_{\text{х.в}}^0$, $I_{\text{г.в}}$
- Коэффициент избытка воздуха $\alpha_{\text{ух}}$
- Доля рециркуляции воздуха β
- Потери от химической неполноты $q_{\text{пл}}$ (%) — задаются вручную

3.3. Котёл

- Номинальная и фактическая паропроизводительность $D_{\text{ном}}$, D (т/ч)
- Ручное переопределение $q_{\text{пл}}_{\text{ном}}$ (иначе — интерполяция по рис. 5.1)
- Площадь неохлаждаемых поверхностей $H_{\text{неохл}}$
- Тепло с подогретым воздухом $Q_{\text{в.вн}}$ (для твёрдого/жидкого топлива)

3.4. Полезное тепло

Параметры формулы 5-16: расходы и энтальпии перегретого пара, питательной воды, непрерывной продувки, пара через пароперегреватель, отборы тепла. Энтальпии $i_{\text{пе}}$, $i_{\text{пв}}$, i' , $i_{\text{ф}}$ можно рассчитать в плагине «Свойства пара» и подставить в форму одной кнопкой.

3.5. Расход топлива

- Переопределение $Q_{\text{к}}$ (по умолчанию берётся $Q_{\text{полн}}$)
- Тепло внешнего воздуха (формула 5-20)
- Паровое дутьё / впрыск (формула 5-21)

3.6. Результаты

- Метрики: $\eta_{\text{пл}}$, $Q_{\text{полн}}$, B , $B_{\text{пл}}$, Σq
- Столбчатая диаграмма статей потерь
- Таблица полей результата и список использованных формул
- Практические рекомендации по режиму работы

3.7. Меню и настройки

Действие	Описание
Сохранить	Запись входных данных (и результата) в JSON v2
Загрузить	Восстановление полей из сохранённого JSON
Утилиты	Открытие плагинов (например, «Свойства пара»)
Настройки	Режим подписей полей; сведения о программе

Файл settings.json: параметр label_mode = full | compact — полные подписи (символ + описание + единица) или компактные (символ + единица).

3.8. Типовой сценарий работы

- Выберите топливо из справочника или введите характеристики вручную
- Заполните уходящие газы и параметры котла
- При необходимости откройте «Свойства пара», рассчитайте энтальпии и подставьте их
- Укажите полезные тепловые нагрузки
- Выполните расчёт и откройте раздел «Результаты»
- При необходимости сохраните сессию в JSON

3.9. Требования к данным

- $Q_{\text{г}}$ и располагаемое тепло должны быть положительными
- $D > 0$ для корректного пересчёта $q_{\text{г}}$ по нагрузке
- Для В нужны $Q_{\text{к}} > 0$ и знаменатель формулы 5-19 $\neq 0$
- При некорректных данных показываются рекомендации, приложение не аварийно завершается

4. Архитектура системы

Приложение разделено на слои: UI → ядро расчёта → данные / плагины. Ядро не зависит от CustomTkinter и может встраиваться в другие системы.

```
main.py
├─ apply_windows_fixes()      # DPI / Win32
├─ JsonFileDataProvider      # data/reference/
├─ PluginRegistry            # plugins/
└─ HeatBalanceApp.mainloop() # CustomTkinter
```

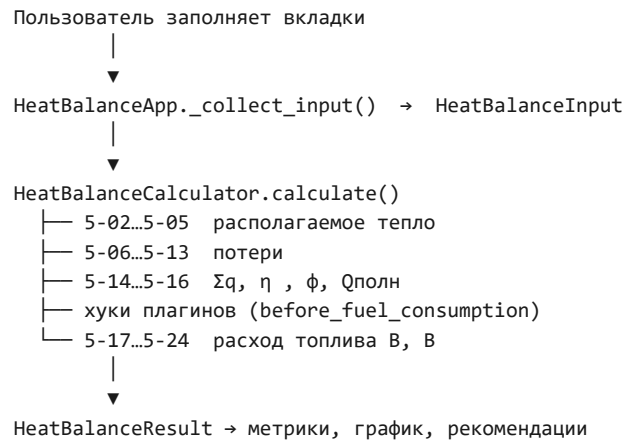
4.1. Слои и модули

Слой	Путь	Назначение
Точка входа	main.py	Инициализация, запуск GUI
Ядро	src/core/	Формулы, модели, таблицы, steam
Данные	src/data/	DataProvider и JSON/БД-провайдеры
API плагинов	src/plugins/	Базовые классы и реестр
Интерфейс	src/ui/	CustomTkinter UI
Утилиты	plugins/	Внешние плагины (steam_quality и др.)
Справочники	data/reference/	fuels.json, q5_nominal.json

Ядро (src/core/)

Модуль	Назначение
heat_balance.py	HeatBalanceCalculator — цепочка формул 5-01...5-24
models.py	Dataclass-модели входа и выхода
tables.py	Кривая q□ном, энтальпия шлака, интерполяция
notation.py	Unicode-обозначения инженерных величин
steam.py	Обёртка IAPWS-IF97

4.2. Поток расчёта



4.3. Провайдеры данных

Класс	Источник
DataProvider	Абстрактный интерфейс
JsonFileDataProvider	data/reference/*.json (по умолчанию)
InMemoryDataProvider	Встроенные значения без файлов
DatabaseDataProvider	Заглушка под будущую БД

4.4. Связь плагина пара с формой

Метод хоста	Назначение
get_steam_form_values()	Чтение текущих энтальпий из формы
apply_steam_values(values)	Подстановка энтальпий в поля
highlight_steam_source_fields(keys)	Подсветка изменённых полей
activate_plugin(plugin_id)	Активация плагина по id

5. Формулы теплового баланса

Реализация: класс `HeatBalanceCalculator` в файле `src/core/heat_balance.py`. Нумерация соответствует главе 5 учебника.

Формула	Что считается	Примечание
5-01	Проверка баланса $Q_{\text{п}} = Q_{\text{т}} + \dots + Q_{\text{л}}$	Информационно
5-02	$Q_{\text{тп}}$ твёрдое/жидкое топливо	<code>FuelType.SOLID_LIQUID</code>
5-02a	$Q_{\text{тп}}$ газообразное	<code>FuelType.GASEOUS</code>
5-03	Физическое тепло $i_{\text{пл}} = c_{\text{пл}} \cdot t_{\text{пл}}$	Опционально
5-04	Тепло на размораживание $\Delta Q_{\text{разм}}$	При $W_{\text{тп}} > W_{\text{лп}}$
5-05	Разложение карбонатов	$40 \cdot (\text{CO}_2)_{\text{карб}}$
5-06	Потери с уходящими газами $q_{\text{г}}$	
5-07	Химическая неполнота $q_{\text{х}}$	Из ввода <code>flue.q3</code>
5-08	$\Delta I_{\text{зл}}$	Поправка золы/уноса
5-09	Механическая неполнота $q_{\text{м}}$	
5-10	Потери в окружение $q_{\text{ок}}$	$q_{\text{ном}} \cdot D_{\text{ном}}/D$
5-11	Коэффициент сохранения тепла ϕ	
5-12	Физическое тепло шлака $q_{\text{ш}}$	
5-13	Потери неохлаждаемых поверхностей	При $H_{\text{неохл}} > 0$
5-14	Сумма потерь Σq	
5-15	КПД $\eta_{\text{к}} = 100 - \Sigma q$	
5-16	Полезное тепло $Q_{\text{полн}}$, кВт	
5-17	Тепло избыточного воздуха «в сторону»	Опционально
5-18	Рециркуляция газов	Опционально
5-19	Расход топлива B	
5-20	Тепло внешнего воздуха $Q_{\text{в.вн}}$	Опционально
5-21	Тепло парового дутья $Q_{\text{ф}}$	Опционально
5-22	Пересчёт сухое $\rightarrow$ рабочее	Опционально
5-23	Коррекция $\eta_{\text{к}}$ (сухое топливо)	Опционально
5-24	Расчётный расход $B_{\text{р}}$	$B \cdot (1 - q_{\text{г}}/100)$

Ключевые соотношения

Располагаемое тепло

Твёрдое / жидкое:

$$Q_{\text{тп}} = Q_{\text{т}} + i_{\text{тл}} + Q_{\text{в.вн}} + \Delta Q_{\text{разм}}$$

Газообразное:

$$Q_{\text{тп}} = Q_{\text{т}} + i_{\text{тл}} + \Delta Q_{\text{разм}}$$

КПД и расход

$$\eta_k = 100 - (q_2 + q_3 + q_4 + q_5 + q_6 + q_{5охл})$$

$$B = Q_k / (Q_{i_p} \cdot \eta_k / 100 + Q_{v_vn} + Q_{\phi})$$

$$B_p = B_{corrected} \cdot (1 - q_4 / 100)$$

Значение $q_{\square ном}$ берётся из кривой рис. 5.1 (линейная интерполяция по $D_{ном}$) или из ручного override. Энтальпия шлака $(с\theta)_{шл}$ — по фрагменту табл. XIV либо из поля ввода. В HeatBalanceResult.formulas_used накапливаются номера формул, реально применённых при текущем наборе опций.

6. Программный API ядра

Ядро в каталоге `src/core` не требует GUI. Подходит для скриптов, тестов и встраивания в SCADA / инженерные системы.

Минимальный пример

```
from src.core.heat_balance import HeatBalanceCalculator
from src.core.models import (
    FuelType, FuelProperties, HeatBalanceInput, UsefulHeatParams,
)

inp = HeatBalanceInput(
    fuel=FuelProperties(
        fuel_type=FuelType.GASEOUS,
        Q_i_p=35588.0, A_p=0.0, W_r_c=0.0,
    ),
    useful=UsefulHeatParams(D_pe=20.0, i_pe=3400.0, i_pv=420.0),
)

result = HeatBalanceCalculator().calculate(inp)
print(result.eta_k, result.Q_poln, result.B)
```

Модели входа

Класс	Содержание
FuelProperties	Тип топлива, Q_i^p , влажность, зольность, карбонаты, $i_{\text{пл}}$
AshLosses	Доли золы, горючее в шлаке/уносе, $t_{\text{шл}}$
FlueGasParams	Энтальпии газов, $\alpha_{\text{ух}}$, β , $q_{\text{г}}$
BoilerParams	$D_{\text{ном}}$, D , $q_{\text{ном}}$ override, $H_{\text{неохл}}$, $Q_{\text{в.вн}}$
UsefulHeatParams	Расходы/энтальпии для $Q_{\text{полн}}$
FuelConsumptionParams	Опции формул 5-17...5-23
HeatBalanceInput	Агрегат всех групп входа

Модель результата HeatBalanceResult

Поле	Ед.	Смысл
$Q_{\text{p.p}}$	кДж/кг(м^3)	Располагаемое тепло
$q_2 \dots q_6_{\text{шл}}$, $q_5_{\text{охл}}$	%	Статьи потерь
sum_q	%	Сумма потерь
eta_k	%	КПД котла
ϕ	—	Коэффициент сохранения тепла
$Q_{\text{полн}}$	кВт	Полезное тепло
B , B_p	кг/с	Расход и расчётный расход
<code>formulas_used</code>	list	Номера применённых формул
<code>warnings</code>	list	Практические рекомендации
<code>plugin_contributions</code>	dict	Данные от плагинов

Сериализация: `result.to_dict()`. Конструктор калькулятора: `HeatBalanceCalculator(data_provider=None, plugin_registry=None)`.

Свойства пара (`src/core/steam.py`)

Независимо от GUI доступны `calculate_state()`, `saturation_by_pressure/temperature()`, `generate_saturation_table()`, `calculate_boiler_cycle()`. Режимы ввода: P-T, P-x, P-h, P-s, T-x, h-s, T-s, насыщение по P/T. Поддерживаются различные единицы давления и температуры.

7. Система плагинов

Плагины расширяют приложение без изменения ядра UI. Внешние плагины лежат в `plugins/<id>/`, API — в `src/plugins/`. При старте реестр сканирует подкаталоги, читает `manifest.json` и загружает модуль через `importlib`.

Манифест

```
{
  "id": "my_plugin",
  "name": "Моя утилита",
  "version": "1.0.0",
  "entry": "plugin.py",
  "description": "Краткое описание"
}
```

Типы плагинов

- `UtilityPlugin` — утилита с собственным окном (`activate`, `open_window`)
- `CalculationPlugin` — расчётный модуль без UI (`calculate`)
- Хук `on_before_fuel_consumption` — вызывается после 5-16, до расчёта В; можно скорректировать `result` или записать `plugin_data`

Шаблон утилиты

```
from src.plugins.base import UtilityPlugin

class MyPlugin(UtilityPlugin):
    id = "my_plugin"
    name = "Моя утилита"
    version = "1.0.0"

    def activate(self, host):
        self._host = host

    def open_window(self, parent):
        ... # CTKTopLevel

def create_plugin():
    return MyPlugin()
```

Фабрика `create_plugin()` обязательна. Ошибки загрузки одного плагина не останавливают приложение.

Встроенный плагин: Свойства пара

Параметр	Значение
Путь	<code>plugins/steam_quality/</code>
Id / версия	<code>steam_quality / 3.1.0</code>
Стандарт	IAPWS-IF97 (пакет <code>iapws</code>)
Вкладки	Состояние, Котёл, Насыщение, Таблицы
Интеграция	Подстановка энтальпий в форму баланса

8. Справочные данные и форматы файлов

8.1. fuels.json

Справочник топлив в data/reference/fuels.json. Поля объекта: id, name, fuel_type (gaseous | solid_liquid), Q_ip, A_p, W_rc, c_{tl}, CO₂carb. Чтобы добавить топливо — допишите объект в массив fuels и перезапустите приложение.

id	Название	Q _i p
natural_gas	Природный газ (ГРС)	35588
associated_gas	Попутный нефтяной газ	41800
lpg	СУГ	46000
fuel_oil_m100	Мазут М-100	40200
fuel_oil_m40	Мазут М-40	41800
diesel	Дизельное топливо	42800
kuzbass_g	Кузнецкий уголь, марка Г	24600
kuzbass_d	Кузнецкий уголь, марка Д	26200
ekibastuz	Экибастузский уголь	19200
gas_coal	Газовый уголь (Г)	31000
hard_coal	Каменный уголь рядовой	22800
anthracite	Антрацит	34000
brown_coal	Бурый уголь	14500
wood_pellets	Древесные пеллеты	17500
petcoke	Нефтяной кокс	32000

Таблица 1. Пресеты топлива (Q_ip в кДж/кг или кДж/м³)

8.2. q5_nominal.json

Кривая рис. 5.1 — пары (D_{ном}, q_{ном}) для линейной интерполяции. Если файл отсутствует, используется встроенная Q5_NOMINAL_CURVE из tables.py.

8.3. Формат сессии расчёта (версия 2)

```
{
  "version": 2,
  "fuel_preset_id": "kuzbass_g",
  "input": {
    "fuel": { }, "ash": { }, "flue": { },
    "boiler": { }, "useful": { }, "consumption": { }
  },
  "result": { "eta_k": 89.5, "B": 2.1 }
}
```

При сохранении всегда пишется input; result — только если расчёт уже выполнялся. При загрузке восстанавливаются поля формы; пересчёт запускается вручную.

8.4. Прочие файлы

- settings.json — UI-настройки (label_mode)

- `assets/logo_header.png` — логотип в шапке (опционально; при отсутствии UI работает без него)

9. Зависимости и структура проекта

```
kotel/
├── main.py           # Точка входа
├── run.bat           # Запуск на Windows
├── requirements.txt
├── settings.json
├── docs/             # Документация (MD + PDF)
├── src/
│   ├── core/         # Расчётный движок
│   ├── data/         # Провайдеры данных
│   ├── plugins/      # API плагинов
│   └── ui/           # Интерфейс CustomTkinter
├── data/reference/   # Справочные JSON
├── plugins/
│   └── steam_quality/ # IAPWS-IF97 калькулятор
```

10. Развитие и ограничения

Планы развития

- Реализация `DatabaseDataProvider` для PostgreSQL/SQLite
- Расширение калькулятора пара (процессы расширения)
- Интеграция в составные SCADA / инженерные системы через API ядра

Текущие ограничения

- Нет автоматизированных тестов
- `DatabaseDataProvider` — заглушка
- Часть опций `FuelConsumptionParams` (избыточный воздух «в сторону», рециркуляция, коррекция сухого топлива) реализована в ядре, но не полностью вынесена в текущий GUI
- Ошибки загрузки отдельных плагинов подавляются в реестре

Конец документа. Актуальные исходные материалы также доступны в файлах `README.md` и `docs/*.md` репозитория проекта.